

Project Atlas: Autonomous Competitive Intelligence Engine

Document Type: Developer Handover & Implementation Guide

Target OS: Ubuntu Linux

Tech Stack: Python (via UV), CrewAI, OpenAI (GPT-4o)

1. Executive Summary (The "Why")

We are building a Minimum Viable Product (MVP) that automates the work of a Market Research Analyst. The application will take a single company name or URL as input, dispatch a team of AI agents to research competitors, analyze their pricing/features, and generate a strategic Markdown report.

Success Criteria:

- Input: "Stripe" (or URL).
 - Process: Fully autonomous (no human-in-the-loop).
 - Output: report_[company]_[date].md containing at least 3 competitors and a SWOT analysis.
-

2. Technical Architecture

2.1 Core Stack

- **Language:** Python 3.10+
- **Package Manager:** uv (Strict requirement: do not use pip/poetry).
- **Framework:** CrewAI.
- **LLM Provider:** OpenAI (Model: gpt-4o). *Note: We are skipping local LLMs to ensure*

reasoning reliability.

2.2 External Tools

- **Search:** SerperDevTool (Google Search API).
 - **Scraping:** FirecrawlScrapeWebsiteTool (Converts websites to LLM-ready Markdown).
-

3. Developer Implementation Guide (Ubuntu)

Phase 1: Environment Setup

Run these commands in your terminal to set up the project structure.

Step 1: Install uv (The Python Package Manager)

Bash

```
# Download and install uv
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Source your shell to update paths (or restart terminal)
source $HOME/.cargo/env
```

Step 2: Initialize Project

We will use the CrewAI CLI (managed by uv) to scaffold the project.

Bash

```
# Install CrewAI CLI as a tool
```

```
uv tool install crewai
```

```
# Create the project scaffold
crewai create crew market_intel
cd market_intel
```

Step 3: Dependency Management

The pyproject.toml file controls dependencies. Ensure you add the tool libraries.

Bash

```
# Add the necessary libraries to your environment
uv add crewai-tools langchain_openai
```

Step 4: Environment Variables

Create a .env file in the root directory.

Bash

```
touch.env
```

Copy/Paste the following into .env:

Ini, TOML

```
OPENAI_API_KEY=sk-proj-...
OPENAI_MODEL_NAME=gpt-4o
SERPER_API_KEY=... # Get from serper.dev
FIRECRAWL_API_KEY=... # Get from firecrawl.dev
```

Phase 2: Agent Configuration ("The Brains")

File: src/market_intel/config/agents.yaml
Replace the default content with this:

YAML

```
market_researcher:
  role: >
    Senior Market Intelligence Researcher
  goal: >
    Conduct exhaustive research on {topic}, identifying key competitors,
    pricing models, and feature sets.
  backstory: >
    You are a meticulous researcher who does not trust assumptions.
    You verify facts by visiting company landing pages. You are expert
    at extracting pricing tiers and specific product features.

strategic_analyst:
  role: >
    Lead Competitive Strategist
  goal: >
    Synthesize raw research data into a strategic SWOT analysis and
    executive report.
  backstory: >
    You are a veteran strategy consultant. You take raw data and find
    the narrative. You focus on "So What?" - why does this data matter?
    You write in professional, executive Markdown format.
```

Phase 3: Task Definition ("The Work")

File: src/market_intel/config/tasks.yaml
Replace the default content with this:

YAML

```
research_task:
  description: >
```

1. Search for the top 3-5 direct competitors to {topic}.
2. For each competitor, find their pricing page and feature list.
3. Extract specific data: Monthly price, User limits, Top 3 features.
4. IMPORTANT: You must use the ScrapeWebsiteTool to verify data.

expected_output: >

A raw data dossier containing a list of competitors with their URL, pricing details, and feature lists.

agent: market_researcher

reporting_task:

description: >

Analyze the data provided by the research_task.

1. Create a comparison table of features.
2. Perform a SWOT analysis for {topic} based on the market.
3. Identify 2 key market gaps.

expected_output: >

A professional Markdown report titled "Competitive Analysis: {topic}".
Must include an Executive Summary, Comparison Table, and SWOT.

agent: strategic_analyst

context: [research_task]

Phase 4: Wiring it Together

File: src/market_intel/crew.py

Update the imports and agent definitions to include tools.

Python

```
from crewai import Agent, Crew, Process, Task
from crewai.project import CrewBase, agent, crew, task
from crewai_tools import SerperDevTool, FirecrawlScrapeWebsiteTool
```

```
@CrewBase
```

```
class MarketIntelCrew:
```

```
    """MarketIntel crew"""
```

```
    agents_config = 'config/agents.yaml'
```

```
    tasks_config = 'config/tasks.yaml'
```

```
@agent
def market_researcher(self) -> Agent:
    return Agent(
        config=self.agents_config['market_researcher'],
        # Assign tools ONLY to the researcher
        tools=,
        verbose=True
    )
```

```
@agent
def strategic_analyst(self) -> Agent:
    return Agent(
        config=self.agents_config['strategic_analyst'],
        verbose=True
    )
```

```
@crew
def crew(self) -> Crew:
    return Crew(
        agents=self.agents,
        tasks=self.tasks,
        process=Process.sequential,
        verbose=True
    )
```

Phase 5: Execution

To run the application:

Bash

```
uv run main.py
```

When prompted, enter the target company (e.g., "Linear App" or "Salesforce").

4. Roadmap & Timeline

Phase	Task	Est. Time	Output
1	Environment Setup	1 Hour	Working uv env, API keys verified.
2	Agent Config	2 Hours	agents.yaml and tasks.yaml populated.
3	Tool Integration	2 Hours	crew.py updated, Firecrawl successfully scraping.
4	Testing & Tuning	3 Hours	3 full runs complete. Report quality verified.
5	Handover	-	Final code pushed to Git.

PM Note: If the researcher agent gets stuck in a loop, check the SerperDevTool results. If it returns too many generic links (like "Top 10 lists"), refine the tasks.yaml prompt to ask for "Official company landing pages."